ARTIFICIAL INTELLIGENCE (E016350)
GHENT UNIVERSITY
2025/2026 AY

Prof. Aleksandra Pizurica
Assistants:
Ide Van den Borre
Roel Huysentruyt

Project: Deep Learning

1 Overview

In this project, you will take the first steps toward becoming a deep learning data scientist. Not only will you apply the deep learning theory learned in the lessons, but you will also practice the implementation, documentation, and validation of your model.

The report should be written in the form of a scientific article, including an introduction, methodology, results & discussion, and conclusion. You may include additional sections and references. The report should not exceed 6 pages (double column, font size between 10 and 12). The recommended LaTeX template is provided.

This project is to be completed in groups as they are listed on Ufora. Each group has to submit its code and report; see Section 8 for instructions.

2 Dependencies

After completing the notebook *0. Introduction.ipynb* from the lab sessions, you should have everything installed to complete this project. As a reminder, the following packages were installed:

- *NumPy*
- *Pandas*
- *Matplotlib*
- *Seaborn*
- *Scikit-learn*
- *SciKeras*
- *PyTorch*
- *SciPy*
- *imbalanced-learn*
- *Pillow*
- *Requests*
- *TensorBoard*

Not all of these packages will necessarily be required for this project.

3 Guidelines

- For deep learning, both PyTorch and TensorFlow can typically be used, as they both contain the required functionalities. PyTorch is often preferred because it is more intuitive and flexible, especially for research and rapid experimentation. Therefore, in this lab, you should also use PyTorch, which is already installed. You can find the basics here: PyTorch Basics.
- Organize your code logically.
- Ensure your plots are clear and include appropriate legends and units. Images should be large enough to be easily visible and should be accompanied by the original label and, during the evaluation phase, their prediction/evaluation.
- Avoid spending excessive time on hyperparameter tuning. Focus on implementing a robust search strategy rather than finding the optimal values.
- Ensure that your model is re-initialized between runs. The easiest way is to instantiate a new model for each run.
- In your report, use a scientific writing style and answer all requested parts.

4 Automated Detection of Knee Osteoarthritis from Radiographs

4.1 Introduction

Osteoarthritis (OA) is one of the most common joint disorders and a major cause of pain, disability, and reduced mobility worldwide. Knee osteoarthritis in particular affects a large portion of the aging population and places a significant burden on healthcare systems. Radiographic imaging is the standard method used to assess disease severity, most commonly using the Kellgren–Lawrence (KL) grading system, which categorizes osteoarthritis from grade 0 (no signs of OA) to grade 4 (severe joint damage).

Despite its widespread use, KL grading is performed manually by clinical experts and can be both time-consuming and subjective. Variability between observers may lead to inconsistent grading, particularly in borderline cases. As the number of medical images continues to grow, these limitations highlight the need for tools that can support clinicians in evaluating radiographs more efficiently and consistently.

Automating KL grading with deep learning offers a promising solution. Deep learning models can learn visual patterns from large collections of radiographs and assist clinicians in detecting structural changes associated with osteoarthritis. The goal of this project is to develop and evaluate a deep learning–based classifier to assist in KL grading of knee osteoarthritis using a real-world dataset of knee radiographs. For this project, the KL grading task is simplified to classifying images as either healthy (grade 0) or osteoarthritis (OA) (grades 3 and 4).

4.2 Data

The data used in this project are a subset of a publicly available dataset hosted on Kaggle[1]. The accompanying `KneeOA.zip` file contains the training, validation, and test sets, which are already organized into separate folders.

The dataset consists of knee radiographs labeled according to the Kellgren–Lawrence grading system. The KL grades are defined as follows:

- Grade 0 (Healthy): Healthy knee image.
- Grade 1 (Doubtful): Doubtful joint space narrowing with possible osteophytic lipping.
- Grade 2 (Minimal): Definite presence of osteophytes and possible joint space narrowing.
- Grade 3 (Moderate): Multiple osteophytes, definite joint space narrowing, with mild sclerosis.
- Grade 4 (Severe): Large osteophytes, significant joint space narrowing, and severe sclerosis.

For this project, the problem is simplified to a binary classification task distinguishing healthy knees from moderate to severe osteoarthritis cases. Therefore:

- Grade 0 $\rightarrow$ folder 0 (Healthy)
- Grades 3 and 4 $\rightarrow$ folder 1 (OA)

Figure 1: Example knee X-ray images from the dataset labeled according to their KL grade. In this project, only grades 0, 3, and 4 are used.

[1] https://www.kaggle.com/datasets/shashwatwork/knee-osteoarthritis-dataset-with-severity

Task 1: Data Exploration, Pre-Processing and Augmentation

In this task, you will create a pipeline to load and transform your training, validation, and test data. Create a Jupyter notebook named `preprocessing.ipynb`, implementing the requirements below.

1. **Loading the data**
   - Create the dataset based on the `KneeOA` folder.
   - Load the images as tensors. Note that the images are provided as `.png` files, which are typically loaded with three channels (RGB). However, since these are grayscale X-ray images, the three channels are identical. Hence, the inputs can be simplified to a single channel. Note that some pretrained models used in Task 3 do require three input channels, so you may have to adapt your dataset later on.
2. **Data Exploration**
   - Print the size of the training, validation, and test sets.
   - Show the distribution of the classes.
   - Plot a few samples from each class with their corresponding labels.
   - Explore and display statistics on the pixel intensities (e.g., pixel average, global average, pixel standard deviation, global standard deviation, ...).
3. **Pre-processing**
   - Check the original image size. Downsample the images by resizing them to 128×128 pixels. This decreases the image resolution and loses some information. Nevertheless, it lowers the computational requirements in future steps.
   - Implement a normalization strategy using fixed values, sample statistics, or the statistics of the training dataset, and test it on a sample from the validation dataset.
4. **Augmentation**
   - Implement at least two different relevant types of data augmentation.
   - Visualize the transformations on a sample from the training dataset, both individually and combined.
5. **Pipeline**
   - Implement the full pipeline for preprocessing and data augmentation for the next tasks. Reflect on whether data augmentation should be applied to the validation and test sets.

In your report, provide a detailed discussion of the dataset and the preprocessing steps you implemented, addressing the following aspects:

- **Dataset:** Clearly discuss the dataset used and its statistics.
- **Classification challenges:** Discuss the intrinsic difficulties of the dataset and its distribution for the classification task. This includes, for example, the observed image variability and class distribution.
- **Image handling:** Discuss how you preprocessed your images. This includes the chosen normalization and data augmentation. Clearly motivate your choices.

Task 2: Building a Simple Baseline Model

In this task, you will build and train a classification model from scratch. This will serve as your baseline model against which you can compare results. Create a Jupyter notebook named `baseline.ipynb`, which implements the following steps (you can import the dataset loading from previous steps into this notebook):

Hint: If your initial training runs result in poor accuracy on the test set, find out why this is the case based on the observations made in Task 1. Carefully reconsider your training and final evaluation strategy to address this problem. In particular, reflect on the implementation of the loss function. A conventional loss may not sufficiently penalize misclassifications. Additionally, you can think about complementary strategies.

Hint: Some loss functions have a built-in activation function. In these cases, you do not need a final activation layer in your network. However, when validating the model, you should then apply the activation to your outputs before performing the classification.

1. **Set up the classification:**
   - Choose one or more metrics to evaluate the success of your classifier.
2. **Build your initial model:**
   - Build a classifier model with a convolutional part followed by a fully connected part. The convolutional part must have convolutional layers, but you can also include activation, pooling, or batch normalization layers.
   - The fully connected part must have fully connected layers and one or more dropout layers.
   - Print the initial architecture.
   - Print the number of trainable parameters.
3. **Train your model:**
   - Choose a good loss function for training the model and reasonable starting hyperparameter values.
   - Decide which performance metric(s) you will use to evaluate the model and briefly justify your choice.
   - Train your initial model on the dataset while monitoring the performance on the validation set.
   - Ensure that the model runs without errors and that the training loss decreases smoothly.
   - Plot the model history containing its performance at each epoch for both the training and validation sets.
4. **Hyperparameter tuning:**
   - Typically, hyperparameter tuning is performed. As this requires training multiple models, which is time-consuming, you do not have to complete a full tuning procedure. Nevertheless, make smart choices for your hyperparameters. Furthermore, you can also test different loss functions or optimizers.

5. **Your final baseline:**

   - Retrain your model using the complete training dataset (including the validation set) with the previously determined optimal set of hyperparameters.
   - Plot the training curve.
   - Save the model and load it again.
   - Evaluate the model on the test dataset and report the performance metric(s) chosen for this problem.
   - Plot a few samples from the test dataset (without normalization) together with their predictions and ground-truth labels.
   - Perform a ROC–AUC analysis and plot the ROC curve.
   - Based on the ROC curve, discuss which classification threshold you would choose and justify your choice.
   - Display the confusion matrix based on the considered decision threshold.
   - Reflect on the clinical implications of this threshold choice (e.g., prioritizing sensitivity vs. specificity).

Discuss the implementation of the baseline model in your report. Include the following:

- **Model architecture:** Describe and discuss your model architecture.
- **Metrics:** Which metrics did you use to evaluate the model? Why are these appropriate, and how do you interpret them based on your results? Also consider the clinical interpretation of the chosen metrics.
- **Training and validation loss:** Which loss function did you choose and why? What conclusions can you draw from the loss curves? Did you observe signs of overfitting? If so, which countermeasures against overfitting did you take or would you suggest?
- **Hint:** If you made any changes based on the hint above, include and motivate them in the sections below.
- **Hyperparameters:** Discuss the chosen hyperparameters.
- **Others:** If you carried out further optimization or experiments, you may include them here as well. For example, did you try other loss functions?
- **Performance:** What conclusions can you draw from the performance on the test set? How does the chosen decision threshold affect the clinical interpretation of the model's predictions?

Task 3: Transfer Learning

In this task, you will apply **transfer learning** using a pretrained ResNet50 model (use `ResNet50_Weights.IMAGENET1K_V2`) trained on ImageNet.

Transfer learning is a powerful machine learning technique for leveraging knowledge learned from one task and applying it to a new task. For instance, if a model has been trained to recognize raccoons, the features it has learned (such as shapes, textures, or patterns) can be useful

for identifying tanukis, animals with similar visual characteristics. This approach is particularly valuable when the available dataset for the new task is limited in size, making it difficult to train a full-scale model from scratch. The typical workflow for transfer learning in deep learning involves several key steps:

1. **Starting with a pretrained model:** The architecture of a model trained on a large dataset is (partially) replicated, and the weights of some of its layers are imported.
2. **Freezing the pretrained layers:** These layers are frozen (i.e., excluded from the back-propagation update) to prevent the information learned from the initial task from being overwritten during the new training process.
3. **Adding new trainable layers:** These layers are introduced on top of the frozen ones and are specifically designed to adapt the previous knowledge to the new task.
4. **Training on the new dataset:** The model with the new layers is then trained using data from the current task.
5. **Fine-tuning (optional):** In some cases, the entire model (or parts of it) is unfrozen and retrained with a low learning rate. This fine-tuning allows the model to adapt more precisely to the new dataset, often resulting in improved performance.

You will use pretrained features from models trained on a large-scale image recognition dataset, study the effect of transfer learning on model performance and training time, and compare it to the baseline CNN model you trained from scratch in the previous section. You can find an example of transfer learning here: Transfer Learning with PyTorch.

Instructions

Create a new notebook *transferlearning.ipynb*. Again, copy the dataset logic from the first task.

1. **Set up the classification:**
   - Choose the same metric(s) for this task as you did in the previous one.
2. **Set up the base model:**
   - Instantiate the model with the ImageNet pretrained weights. Do not include the final fully connected classification layer.
     Note: this is the last layer of the network.
   - Freeze all layers in the base model.
   - Add one or more fully connected layers on top of the base model to fit the classification task.
   - Add one dropout layer before the last layer(s). Optionally, add more.
   - Print the final architecture.
3. **Train your model:**
   - For the initial model, start with the best hyperparameters found while optimizing the baseline (when applicable).

- Train your initial model using the previously selected loss function and evaluate the training and validation losses.
- Evaluate the model using the same performance metric(s) chosen in Task 2.
- Ensure that the model runs without errors and that the loss decreases more or less smoothly.
- Plot the training and validation curves.
- Again, tune your hyperparameters in a systematic way, but you do not need to perform a full-scale analysis.

4. **Train your model on the full dataset:**

   - Retrain your model using the complete training dataset (including the validation dataset), using your previous set of optimal hyperparameters.
   - Plot the training curve.
   - Save and reload your model.

5. **Fine-tuning the entire model:**

   - Unfreeze all or part of the base model and train the whole model end-to-end for the same number of epochs as you trained the pretrained model.
   - If you notice a sudden drop in performance on the **training** dataset, reload the model and try again with a smaller learning rate. If there is no sign of improvement, skip this step and use the saved model from the previous step in the following parts. Save the best model.
   - Test the saved model on the test dataset and print the performance metric(s) you chose for this problem.
   - Display the confusion matrix.
   - Plot a few samples from the test dataset (without preprocessing) with their predictions (after preprocessing).

In your report, describe the transfer learning you performed, including the following:

- **Model:** How did you choose the architecture for the final layers you added to the pretrained model?
- **Performance:** How did the pretrained model's training curve compare to the final baseline model?
- **Fine-tuning:** Describe any hyperparameters you changed or further fine-tuning you performed.
- **Transfer Learning:** Discuss, based on this experience, the advantages and disadvantages of this approach.

Task 4: Latent Space Analysis and Error Inspection

In this task, you will analyze the internal feature representation learned by the best model obtained in Task 3. Deep neural networks transform input images into increasingly abstract representations through multiple layers. The output of the final hidden layer before the classification layer can be interpreted as a *latent feature representation* of the images.

By visualizing this latent space, it is possible to gain insights into how the model organizes the data and why certain samples may be misclassified.

Create a new notebook named `latentspace.ipynb`. Use the best performing model from Task 3.

Instructions

1. **Load the trained model**
   - Load the best model obtained.
   - Ensure the model is set to evaluation mode.
   - Modify the network so that it outputs the feature vector from the last hidden layer instead of the final classification layer.
2. **Extract latent representations**
   - Pass all images from the validation or test dataset through the model.
   - For each image, extract the latent feature vector from the last hidden layer.
   - Store for each sample:
     - the (flattened) latent feature vector,
     - the ground-truth label,
     - the predicted label,
     - the prediction confidence.
3. **Dimensionality reduction**
   - Since the latent vectors are high-dimensional, reduce their dimensionality to two dimensions for visualization.
   - Apply the following methods:
     - **PCA**
     - **t-SNE**
   - Plot the 2D latent space obtained with both methods.
   - Color the samples according to their ground-truth class label.
4. **Comparison of PCA and t-SNE**
   - Compare the latent space structure obtained with PCA and t-SNE.
   - Investigate whether the classes appear more clearly separated with one method than the other.

5. **Inspection of misclassified samples**
   - Display several misclassified test images along with:
     - their ground-truth label,
     - the predicted label,
     - the prediction confidence.
   - Compare these samples with nearby correctly classified samples in the latent space.

In your report, discuss the results of this analysis and address the following points:

- **Latent space structure:** Do healthy knees and severe osteoarthritis cases form distinct clusters in the latent space?
- **Dimensionality reduction methods:** What differences do you observe between the PCA and t-SNE visualizations? Describe your observations regarding the latent space structure.
- **Misclassified samples:** Where do misclassified samples appear in the latent space?
- **Interpretation:** What can these observations tell you about the model's behavior and the intrinsic difficulty of the classification task?

Task 5: Model Explainability with Grad-CAM

In this task, you will investigate whether a supervised classification model focuses on clinically relevant regions of the knee X-ray images when predicting the osteoarthritis class. To this end, you will use **Gradient-weighted Class Activation Mapping (Grad-CAM)** [1], a visualization technique that highlights image regions that contribute most strongly to the model's prediction.

Interpretability is particularly important in medical imaging, where high classification performance alone is not sufficient. In addition to predicting the correct label, it is desirable that the model bases its decision on meaningful anatomical structures, such as the joint space, osteophyte regions, or other visible signs of severe osteoarthritis. Furthermore, interpretability can provide insight into why the model fails on specific cases.

Create a new notebook named `GradCAM.ipynb`. Use one of your previously trained supervised models from Task 2 or Task 3.

Instructions

1. **Select a trained classifier**
   - Choose your best performing model from one of the previous tasks.
   - Clearly indicate which model you use for this analysis.
   - Reload the trained model and verify that it produces the expected predictions on the test set.
2. **Implement Grad-CAM**
   - Apply Grad-CAM to a suitable convolutional layer of the selected model.
   - Generate Grad-CAM heatmaps for a number of test images.
   - Overlay the heatmaps on the original X-ray images for visualization.
3. **Qualitative analysis**
   - Visualize Grad-CAM results for at least a few correctly classified healthy images and a few correctly classified severe osteoarthritis images.
   - Additionally, visualize Grad-CAM results for some misclassified examples.
   - Compare whether the highlighted regions differ between healthy and severe cases.
4. **Clinical interpretation**
   - Analyze whether the model appears to focus on clinically meaningful regions, such as the knee joint space.
   - Discuss whether the highlighted regions are plausible from a medical imaging perspective.
   - Investigate whether there are examples where the model seems to focus on irrelevant structures or image artifacts.

In your report, briefly discuss the following aspects:

- **Interpretability method:** Describe the Grad-CAM method and explain on which model and layer it was applied.
- **Qualitative findings:** Summarize the main observations from the Grad-CAM visualizations.
- **Clinical relevance:** Discuss whether the model focuses on anatomically and clinically plausible regions.
- **Limitations:** Reflect on the limitations of Grad-CAM as an interpretability tool and explain why such visualizations should be interpreted with caution.

5 Report

Your report should be written in the style of a two-column article with a font size between 10 and 12, not exceeding 6 pages. If you require more space, you can add more details in an Appendix if needed (e.g., loss curves, model architectures, hyperparameter tuning ...). Please use the LaTeX template available in the Ufora content section dedicated to the project.

The report must include:

- Introduction
- Methodology sections corresponding to each of the specified tasks
- Results & Discussion
- Conclusion
- Author contributions and collaboration (see description further on)
- Use of generative AI (see description further on)

Ensure that the report answers all questions provided in the tasks. Use a technical writing style (concise, precise, and with correct terminology/notation) and follow a professional layout. Avoid verbose and vague descriptions.

6 Author contributions and collaboration

Describe here how you collaborated on the project and specify for each group member his or her concrete contributions. Each group member bears responsibility for all parts of the submitted code and report, including possible misconduct in the parts assigned to the other partners. In addition, every group member is expected to be familiar with all parts of the code and all sections of the report, not only with the parts to which they contributed the most. This means that each member should understand the complete project and be able to clearly explain any part of the implementation, analysis, and written report if required.

7 Use of Generative AI

The use of generative AI has become widespread and can significantly accelerate both coding and scientific writing. You are therefore allowed to use generative AI tools for implementing code and drafting your report. However, the primary objective of this project is to develop a solid understanding of deep learning workflows. From a pedagogical perspective, meaningful learning occurs when you implement and reason through the key steps yourself.

If you choose to use AI-generated code, you remain fully responsible for verifying its correctness for this specific project. In particular, pay attention to the following aspects:

- **Code:** Generative AI tools may produce unnecessarily complex solutions or include components that are outside the scope of the assignment. Ensure that your implementation directly addresses the questions in a concise manner. Be critical of suggested design choices (e.g., data augmentation strategies) and verify that they are clinically and methodologically justified for this dataset.

- **Writing:** AI-generated text often contains generic statements that lack critical reflection or dataset-specific insights. Avoid using AI to generate full sections from scratch. Instead, use it primarily as a tool for language refinement.

In your report, clearly state whether you used generative AI tools. If so, briefly describe how they were used. We encourage thoughtful and transparent use of generative AI. However, the following will be penalized:

- Inclusion of code that is outside the defined project scope or is clearly not understood by the student.
- Long, generic answers that are not grounded in your own experiments, observations, and critical analysis.

8 Submission

8.1 Submission Files

Submit two separate files:

- A zipped package containing your code (notebooks). Exclude the dataset and models.
- A PDF report, max. 6 pages, with additional information in the appendix allowed, formatted as an article using the provided template.

Name your ZIP file as
`Codes_group{#Number}.zip` (e.g., `Codes_group21.zip`).
and your PDF report as
`Report_group{#Number}.pdf` (e.g., `Report_group21.pdf`).

8.2 Submission Instructions

- **Submission Method:** Upload the files to Ufora via Ufora-tools → Assignments → Project 1: Deep Learning.
- Submit the code and report as two separate files.
- **Deadline:** 30/04/2026, 23:59 CET.

8.3 Evaluation Criteria

The final grade is based on (in order of importance):

1. Methodology.
2. Report quality.
3. Code quality.
4. Model performance.

Important Remarks:

- Data leakage (directly or indirectly using test data information in training) results in fewer points.
- Report quality includes cohesion and overall consistency.
- Poor English, grammatical errors, and unclear sentences lower the report quality score.
- The presence and quality of plots and images affect the report score.
- References to external sources are optional. Use them sparingly, primarily for context. While demonstrating familiarity with related academic works may improve your report quality, we discourage devoting too much time to this. In particular, we do not expect you to acquire deep medical knowledge or reproduce a state-of-the-art model.
- Code readability affects the score. Add comments for non-obvious or non-standard sections (not for every line).
- Generative AI is allowed, but its misuse will lower your score (see Section 7).
- Plagiarism will have repercussions beyond this project and course. Share advice with other groups, not code or the report.

For technical queries, use the Ufora forum (Ufora → Ufora-tools → Discussions → Project: Medical Image Classification). Anonymous questions are allowed.

References

[1] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," *International Journal of Computer Vision*, vol. 128, no. 2, pp. 336–359, 2020.